

Modul Praktikum
Pemrograman Berorientasi Objek (PBO)
“Exception Handling”

Minggu / Sesi	:	VI / 2-3
Tujuan	:	Mahasiswa mampu memahami dan menerapkan Exception Handling dalam pemrograman Java
Setoran	:	Softcopy
Waktu Penyetoran	:	Akhir Sesi
Tempat Setoran	:	ecourse.del.ac.id

A. PEMAHAMAN

1. Apa itu Exception?

Exception adalah kejadian *tidak normal (error)* yang terjadi selama program dijalankan (*runtime*) dan mengganggu alur eksekusi normal program. Kata "exception" berarti "pengecualian" — yaitu kondisi di luar alur yang diharapkan programmer.

Contoh sederhana exception dalam kehidupan sehari-hari:

- Mencoba membagi angka dengan nol → `ArithmeticException`
- Mengakses elemen array pada indeks yang tidak ada → `ArrayIndexOutOfBoundsException`
- Membuka file yang tidak ditemukan → `FileNotFoundException`
- Mengakses objek yang bernilai null → `NullPointerException`

Anatomi Exception di Java

Throwable (induk semua error & exception)

- └─ Error (masalah serius, tidak perlu ditangani programmer)
- └─ Exception (masalah yang BISA dan HARUS ditangani)
 - └─ Checked Exception (diperiksa saat kompilasi)
 - └─ Unchecked Exception (terjadi saat runtime / `RuntimeException`)

2. Mengapa Exception Handling Penting?

Tanpa exception handling, program akan langsung berhenti (crash) saat mengalami error. Exception handling membuat program lebih:

- **Robust (Tangguh)** — Program tetap berjalan meski terjadi error; tidak langsung crash.
- **User-Friendly** — Pengguna mendapat pesan error yang informatif, bukan pesan teknis yang membingungkan.
- **Maintainable (Mudah Dipelihara)** — Error tercatat (log) sehingga mudah ditemukan dan diperbaiki.

- **Secure (Aman)** — Mencegah kebocoran informasi sensitif sistem ke pengguna.

Sintaks dasar exception handling di Java menggunakan blok try-catch-finally:

```
try {
    // Kode yang mungkin melempar exception
} catch (NamaException e) {
    // Penanganan jika exception terjadi
} finally {
    // Selalu dieksekusi, baik exception terjadi atau tidak
}
```

3. Jenis-Jenis Exception

Java mengelompokkan exception menjadi dua kategori utama, yaitu Checked Exception dan Unchecked Exception.

a) Checked Exception

Checked Exception adalah exception yang **diperiksa oleh compiler pada saat kompilasi**. Programmer WAJIB menangani atau mendeklarasikannya menggunakan kata kunci **throws**. Jika tidak, program tidak bisa dikompilasi.

- IOException — Kesalahan operasi input/output (membaca/menulis file)
- SQLException — Kesalahan operasi database
- FileNotFoundException — File yang dicari tidak ditemukan

b) Unchecked Exception (RuntimeException)

Unchecked Exception adalah exception yang **baru terdeteksi saat program dijalankan** (runtime). Compiler tidak memaksa programmer untuk menanganinya, namun tetap penting untuk diantisipasi.

- ArithmeticException — Pembagian dengan nol
- NullPointerException — Mengakses objek yang bernilai null
- ArrayIndexOutOfBoundsException — Indeks array di luar batas
- NumberFormatException — Konversi string ke angka yang tidak valid

c) Custom Exception

Custom Exception adalah exception **buatan programmer sendiri** untuk kebutuhan logika bisnis spesifik yang tidak tersedia di Java. Dibuat dengan cara meng-extend kelas **Exception** atau **RuntimeException**.

Jenis Exception	Kategori	Contoh Kelas	Wajib Ditangani?
IOException	Checked	FileNotFoundException	Ya
SQLException	Checked	SQLSyntaxErrorException	Ya
ArithmeticException	Unchecked	RuntimeException	Tidak (opsional)
NullPointerException	Unchecked	RuntimeException	Tidak (opsional)
ArrayIndexOutOfBoundsException	Unchecked	RuntimeException	Tidak (opsional)
NumberFormatException	Unchecked	RuntimeException	Tidak (opsional)
Custom Exception	Checked / Unchecked	Dibuat sendiri	Sesuai pilihan

B. PROSEDUR PRAKTIKUM

Buatlah folder dengan nama **w06_pbo_nim** dan simpan semua kode program pada praktikum sesi ini ke dalam folder tersebut (*nim diganti dengan NIM Anda*).

Latihan 1 — Checked Exception & Unchecked Exception

File: BasicException.java

Topik: Try-Catch-Finally, IOException, ArithmeticException

1

Pada folder w06_pbo_nim, buatlah file baru dengan nama BasicException.java dan masukkan kode program berikut.

```
1  import java.io.*;
2
3  // =====
4  // Class BasicException - Demonstrasi Checked dan Unchecked Exception
5  // =====
6  public class BasicException {
7
8      // Implementasi Checked Exception dengan IOException
9      // Dideklarasikan dengan 'throws' pada method readFile()
10     public static void readFile(String filename) throws IOException {
11         FileReader fr = new FileReader(filename); // jika file tidak ada -> IOException
12         System.out.println("File opened successfully: " + filename);
13         fr.close();
14     }
15
16     // Implementasi Unchecked Exception dengan ArithmeticException (subclass RuntimeException)
17     // Dideklarasikan, namun tetap harus diantisipasi oleh programmer.
18     public static int arithmeticDivision(int a, int b) {
19         if (b == 0) {
20             // melempar exception jika b = 0 untuk mencegah pembagian dengan nol
21             throw new ArithmeticException("Division by zero is not allowed.");
22         }
23         return a / b;
24     }
25
26     // Method main() sebagai titik masuk program.
27     public static void main(String[] args) {
28
29         // --- Percobaan 1: Menangani Checked Exception ---
30         System.out.println("=== Demo Checked Exception ===");
31         try {
32             // Mencoba membuka file yang tidak ada
33             readFile("data.txt");
34         } catch (IOException e) {
35             // Blok catch menangkap IOException dan menampilkan pesan
36             System.out.println("[CATCH] An IOException has been caught: " + e.getMessage());
37         } finally {
38             // Blok finally selalu dieksekusi, berhasil atau tidak
39             System.out.println("[FINALLY] Finally block always executes.");
40         }
41
42         // --- Percobaan 2: Menangani Unchecked Exception ---
43         System.out.println("\n=== Demo Unchecked Exception ===");
44         try {
45             int result = arithmeticDivision(10, 0); // sengaja dibagi nol
46             System.out.println("Result: " + result);
47         } catch (ArithmeticException e) {
48             System.out.println("[CATCH] ArithmeticException: " + e.getMessage());
49         }
50
51         System.out.println("\nProgram Finished.");
52     }
53 }
54
```

Penjelasan Kode Program BasicException.java

Berikut adalah penjelasan detail setiap bagian dari kode program di atas:

- **Baris 1:** `import java.io.*` mengimpor semua kelas dari paket `java.io`, termasuk `FileReader` dan `IOException` yang dibutuhkan untuk operasi file.
- **Baris 10–14 (Method `readFile`):** Deklarasi method yang menerima parameter `String filename`. Kata kunci `throws IOException` mendeklarasikan bahwa method ini dapat melempar Checked Exception bertipe `IOException`. Jika file tidak ditemukan, Java otomatis melempar `IOException`.
- **Baris 18–24 (Method `arithmeticDivision`):** Method yang menangani pembagian bilangan. Menggunakan `throw new ArithmeticException()` untuk secara eksplisit melempar Unchecked Exception ketika nilai `b` bernilai 0. Hal ini karena memberikan pesan error yang lebih informatif dibanding membiarkan Java Virtual Machine melemparnya secara otomatis.
- **Baris 30–40 (Percobaan 1 — Checked Exception):** Blok `try` mencoba memanggil `readFile()` dengan nama file yang tidak ada. Blok `catch (IOException e)` menangkap exception dan menampilkan pesan melalui `e.getMessage()`. Blok `finally` memastikan pesan konfirmasi selalu tercetak meskipun exception terjadi.
- **Baris 43–49 (Percobaan 2 — Unchecked Exception):** Blok `try` memanggil `arithmeticDivision (10, 0)`. Karena nilai pembagi adalah 0, method melempar `ArithmeticException`. Blok `catch` menangkap exception tersebut dan menampilkan pesannya.

2

Buka terminal atau command prompt, arahkan ke folder `BasicException.java`, lalu lakukan kompilasi dan eksekusi dengan perintah:

```
javac BasicException.java
java BasicException
```

```
C:\Users\KLASH\OneDrive - Institut Teknologi Del\IT Del\KULIAH\2025_2026\SEM_GENAP\4232101_PBO\[3]_Materi_Praktikum\Week_06\w06_pbo_nim>javac BasicException.java
C:\Users\KLASH\OneDrive - Institut Teknologi Del\IT Del\KULIAH\2025_2026\SEM_GENAP\4232101_PBO\[3]_Materi_Praktikum\Week_06\w06_pbo_nim>java BasicException
=== Demo Checked Exception ===
[CATCH] An IOException has been caught: data.txt (The system cannot find the file specified)
[FINALLY] Finally block always executes.

=== Demo Unchecked Exception ===
[CATCH] ArithmeticException: Division by zero is not allowed.

Program Finished.
```

Latihan 2 — Multiple Catch dan Finally

File: MultipleCatch.java

Topik: Multiple Catch Block, ArrayIndexOutOfBoundsException, ArithmeticException

1

Buatlah file baru dengan nama MultipleCatch.java dalam folder yang sama dan masukkan kode program berikut.

```
1 // =====
2 // Class MultipleCatch - Demonstrasi Multiple Catch dan Finally
3 // =====
4 public class MultipleCatch {
5
6     // Method ini berpotensi melempar lebih dari satu jenis exception.
7     public static double processData(int[] data, int index) {
8         // Jika index melebihi panjang array -> ArrayIndexOutOfBoundsException
9         int value = data[index];
10        // Jika nilai adalah 0, pembagian akan gagal -> ArithmeticException
11        return 100 / value;
12    }
13
14    public static void main(String[] args) {
15        int[] arrayNumber = {5, 0, 2}; // array berisi tiga elemen
16
17        // Tiga skenario pengujian dengan multiple catch.
18
19        // Skenario A: Normal (index=0, nilai=5 -> 100/5 = 20)
20        System.out.println("--- Skenario A: Normal ---");
21        try {
22            System.out.println("Result: " + processData(arrayNumber, 0));
23        } catch (ArrayIndexOutOfBoundsException e) {
24            System.out.println("[CATCH] Index out of bounds: " + e.getMessage());
25        } catch (ArithmeticException e) {
26            System.out.println("[CATCH] Division by zero: " + e.getMessage());
27        } finally {
28            System.out.println("[FINALLY] Scenario A finished.");
29        }
30
31        // Skenario B: ArithmeticException (index=1, nilai=0 -> 100/0)
32        System.out.println("\n--- Skenario B: Nilai Nol ---");
33        try {
34            System.out.println("Result: " + processData(arrayNumber, 1));
35        } catch (ArrayIndexOutOfBoundsException e) {
36            System.out.println("[CATCH] Index out of bounds: " + e.getMessage());
37        } catch (ArithmeticException e) {
38            System.out.println("[CATCH] Division by zero: " + e.getMessage());
39        } finally {
40            System.out.println("[FINALLY] Scenario B finished.");
41        }
42
43        // Skenario C: ArrayIndexOutOfBoundsException (index=5 tidak ada)
44        System.out.println("\n--- Skenario C: Index Salah ---");
45        try {
46            System.out.println("Result: " + processData(arrayNumber, 5));
47        } catch (ArrayIndexOutOfBoundsException e) {
48            System.out.println("[CATCH] Index out of bounds: " + e.getMessage());
49        } catch (ArithmeticException e) {
50            System.out.println("[CATCH] Division by zero: " + e.getMessage());
51        } finally {
52            System.out.println("[FINALLY] Scenario C finished.");
53        }
54    }
55 }
56
```

Penjelasan Kode Program MultipleCatch.java

- **Baris 7–12 (Method processData):** Method menerima dua parameter: array integer data dan indeks index. Ada dua titik potensi exception:
 - (1) akses data[index] bisa melempar `ArrayIndexOutOfBoundsException` jika index tidak valid
 - (2) pembagian 100/nilai bisa melempar `ArithmeticException` jika nilai adalah 0.
- **Baris 15 (Deklarasi Array):** Array angka berisi tiga elemen {5, 0, 2}. Elemen indeks 0 (nilai 5) normal, elemen indeks 1 (nilai 0) akan menyebabkan `ArithmeticException`, dan indeks 5 tidak ada sehingga akan menyebabkan `ArrayIndexOutOfBoundsException`.
- **Baris 20-29 (Skenario A):** Pengujian normal dengan indeks 0 dan nilai 5. Hasil $100/5 = 20$, tidak ada exception yang terjadi. Blok finally tetap dieksekusi.
- **Baris 32–41 (Skenario B):** Pengujian dengan indeks 1 dan nilai 0. Pembagian $100/0$ melempar `ArithmeticException`. Blok catch kedua (`ArithmeticException`) yang aktif menangkap exception tersebut.
- **Baris 44–53 (Skenario C):** Pengujian dengan indeks 5 yang tidak ada pada array berukuran 3. Akses data[5] langsung melempar `ArrayIndexOutOfBoundsException`. Blok catch pertama yang aktif menangkap exception tersebut.
- **Konsep Multiple Catch:** Java memeriksa blok catch dari atas ke bawah secara berurutan. Blok catch yang pertama cocok akan dieksekusi, sedangkan yang lain dilewati. Pastikan urutan catch dari yang paling spesifik ke yang paling umum.

2

Kompilasi dan jalankan MultipleCatch.java dengan perintah:

```
javac MultipleCatch.java
```

```
java MultipleCatch
```

Perhatikan perbedaan output dari setiap skenario A, B, dan C.

```
C:\Users\KLASH\OneDrive - Institut Teknologi Del\IT Del\KULIAH\2025_2026\SEM_GENAP\4232101_PBO\[3]_Materi_Praktikum\Week_06\w06_pbo_nim>javac MultipleCatch.java
C:\Users\KLASH\OneDrive - Institut Teknologi Del\IT Del\KULIAH\2025_2026\SEM_GENAP\4232101_PBO\[3]_Materi_Praktikum\Week_06\w06_pbo_nim>java MultipleCatch
--- Skenario A: Normal ---
Result: 20.0
[FINALLY] Scenario A finished.

--- Skenario B: Nilai Nol ---
[CATCH] Division by zero: / by zero
[FINALLY] Scenario B finished.

--- Skenario C: Index Salah ---
[CATCH] Index out of bounds: Index 5 out of bounds for length 3
[FINALLY] Scenario C finished.
```

Latihan 3 — Custom Exception

File: CustomException.java

Topik: Membuat Custom Exception dengan meng-extend kelas Exception

1

Buatlah file baru dengan nama CustomException.java dalam folder yang sama dan masukkan kode program berikut.

```
1 // =====
2 // Class CustomException - Custom Exception modifikasi sendiri
3 // =====
4
5 // Custom Exception harus meng-extend Exception (checked) atau RuntimeException (unchecked).
6 // Di sini kita pilih Exception agar pemanggilnya wajib menangani.
7 class CheckBalanceException extends Exception {
8     // Konstruktor menerima pesan kesalahan yang informatif
9     public CheckBalanceException(String message) {
10         super(message); // Meneruskan pesan ke konstruktor Exception induk
11     }
12 }
13
14 // Class CustomException - Penggunaan Custom Exception
15 public class CustomException {
16
17     // Deklarasi atribut untuk menyimpan nama pemilik rekening dan saldo
18     private String customer;
19     private double balance;
20
21     // Konstruktor dengan parameter untuk inisialisasi customer dan saldo rekening
22     public CustomException(String customer, double balance) {
23         this.customer = customer;
24         this.balance = balance;
25     }
26
27     // Method withdraw() melakukan penarikan saldo.
28     // Jika saldo tidak mencukupi, method melempar Custom Exception.
29     public void withdraw(double amount) throws CheckBalanceException {
30         if (amount > balance) {
31             // Membuat dan melempar custom exception dengan pesan detail
32             throw new CheckBalanceException(
33                 "Penarikan Rp" + amount + " gagal. Saldo saat ini: Rp" + balance
34             );
35         }
36         balance -= amount; // Kurangi saldo jika mencukupi
37         System.out.println("Penarikan Rp" + amount + " berhasil. Sisa saldo: Rp" + balance);
38     }
39 }
```



```

40 // Method accountInfo() menampilkan data rekening.
41 public void accountInfo() {
42     System.out.println("Rekening " + customer + " | Saldo: Rp" + balance);
43 }
44
45 public static void main(String[] args) {
46     // Membuat objek rekening dengan saldo awal Rp 500.000
47     CustomException rekening = new CustomException("Gloria Yuni", 500000);
48     rekening.accountInfo();
49
50     // Percobaan 1: Penarikan yang berhasil
51     System.out.println("\n--- Penarikan Rp200.000 ---");
52     try {
53         rekening.withdraw(200000);
54     } catch (CheckBalanceException e) {
55         System.out.println("[CATCH] " + e.getMessage());
56     }
57
58     // Percobaan 2: Penarikan melebihi saldo -> Custom Exception aktif
59     System.out.println("\n--- Penarikan Rp400.000 ---");
60     try {
61         rekening.withdraw(400000);
62     } catch (CheckBalanceException e) {
63         System.out.println("[CATCH] Custom Exception: " + e.getMessage());
64     }
65 }
66 }
67

```

Penjelasan Kode Program CustomException.java

- **Baris 7–12 (Kelas CheckBalanceException):** Deklarasi Custom Exception dengan cara meng-extend kelas Exception. Karena meng-extend Exception (bukan RuntimeException), ini termasuk Checked Exception sehingga pemanggilnya wajib menangani. Konstruktor memanggil super(message) untuk meneruskan pesan ke kelas induk.
- **Baris 18–19 (Deklarasi Atribut):** Atribut customer (String) dan balance (double) dideklarasikan private untuk menerapkan prinsip encapsulation. Data hanya bisa diakses dan dimodifikasi melalui method yang tersedia.
- **Baris 22–25 (Konstruktor):** Konstruktor menginisialisasi atribut pemilik dan saldo dengan nilai yang diberikan saat pembuatan objek menggunakan kata kunci new.
- **Baris 29–38 (Method withdraw):** Method dideklarasikan dengan throws CheckBalanceException karena menggunakan Custom Exception bertipe checked. Kondisi if (jumlah > saldo) memicu pelemparan exception dengan new CheckBalanceException() beserta pesan ke customer. Jika saldo mencukupi, nilai saldo dikurangi dan pesan sukses ditampilkan.
- **Baris 45–65 (Method main):** Membuat objek rekening dengan saldo awal Rp500.000. Percobaan pertama menarik Rp200.000 (berhasil, sisa saldo Rp300.000). Percobaan kedua menarik Rp400.000 (gagal, saldo tidak mencukupi) sehingga Custom Exception dilempar dan ditangkap oleh blok catch.

2

Kompilasi dan jalankan CustomException.java dengan perintah:

javac CustomException.java

java CustomException

Amati bagaimana Custom Exception memberikan pesan error yang lebih deskriptif dibandingkan exception bawaan Java.

```
C:\Users\KLASH\OneDrive - Institut Teknologi Del\IT Del\KULIAH\2025_2026\SEM_GENAP\4232101_PBO\[3]_Materi_Praktikum\Week_06\w06_pbo_nim>javac CustomException.java
C:\Users\KLASH\OneDrive - Institut Teknologi Del\IT Del\KULIAH\2025_2026\SEM_GENAP\4232101_PBO\[3]_Materi_Praktikum\Week_06\w06_pbo_nim>java CustomException
Rekening Gloria Yuni | Saldo: Rp500000.0

--- Penarikan Rp200.000 ---
Penarikan Rp200000.0 berhasil. Sisa saldo: Rp300000.0

--- Penarikan Rp400.000 ---
[CATCH] Custom Exception: Penarikan Rp400000.0 gagal. Saldo saat ini: Rp300000.0
```

C. TUGAS PRAKTIKUM

Deskripsi Tugas

Pada tugas ini Anda akan membuat sistem manajemen nilai mahasiswa yang menerapkan konsep Exception Handling secara menyeluruh, meliputi Checked Exception, Unchecked Exception, Multiple Catch, dan Custom Exception.

Spesifikasi Program

Buatlah folder bernama **tugas_w06_nim.zip** yang berisi file bernama **NilaiMahasiswa.java**. Program terdiri dari satu class dengan spesifikasi berikut:

a) Custom Exception

- **NilaiTidakValidException** — Dilempar ketika nilai yang dimasukkan di luar rentang 0–100.
- **MahasiswaTidakDitemukanException** — Dilempar ketika data mahasiswa yang dicari tidak ada.

b) Atribut

- **nim**: String untuk Nomor Induk Mahasiswa
- **nama**: String untuk nama lengkap mahasiswa
- **nilaiUTS**: double untuk nilai Ujian Tengah Semester (0–100)
- **nilaiUAS**: double untuk nilai Ujian Akhir Semester (0–100)
- **nilaiTugas**: double untuk nilai rata-rata tugas (0–100)

c) Method yang Wajib Dibuat

1. **setNilaiUTS(double nilai) throws NilaiTidakValidException**
 - Validasi: jika nilai < 0 atau nilai > 100, lempar NilaiTidakValidException
 - Jika valid, simpan nilai ke atribut nilaiUTS
2. **setNilaiUAS(double nilai) throws NilaiTidakValidException**
 - Validasi yang sama dengan setNilaiUTS()
3. **setNilaiTugas(double nilai) throws NilaiTidakValidException**
 - Validasi yang sama dengan setNilaiUTS()
4. **hitungNilaiAkhir(): double**
 - Hitung nilai akhir dengan rumus: $(\text{nilaiUTS} \times 30\%) + (\text{nilaiUAS} \times 50\%) + (\text{nilaiTugas} \times 20\%)$
5. **getGrade(): String**
 - Kembalikan grade berdasarkan nilai akhir: A (≥ 85), B (70–84), C (55–69), D (40–54), E (<40)
6. **tampilkanRaport(): void**
 - Tampilkan informasi lengkap: NIM, nama, semua nilai, nilai akhir, dan grade mahasiswa

7. cariMahasiswa(NilaiMahasiswa[] daftar, String nimCari): NilaiMahasiswa throws MahasiswaTidakDitemukanException

- Mencari mahasiswa dalam array berdasarkan NIM
- Jika tidak ditemukan, lempar MahasiswaTidakDitemukanException

d) Method main() — Skenario Pengujian

Pada method main(), program harus menjalankan minimal empat skenario pengujian berikut:

- **Skenario 1 (Normal):** Membuat minimal 3 objek NilaiMahasiswa, mengisi semua nilai dengan data yang valid, menghitung dan menampilkan raport setiap mahasiswa.
- **Skenario 2 (NilaiTidakValidException):** Mencoba mengisi nilai UAS dengan angka 110 atau angka negatif. Tangkap exception dan tampilkan pesan yang informatif.
- **Skenario 3 (MahasiswaTidakDitemukanException):** Mencoba mencari NIM yang tidak terdaftar dalam array. Tangkap exception dan tampilkan pesan yang informatif.
- **Skenario 4 (ArrayIndexOutOfBoundsException):** Mencoba mengakses elemen array mahasiswa pada indeks yang tidak ada. Tangkap exception menggunakan multiple catch.